Econ 814 Recitation 4 Notes

Frank Gentile

March 8, 2024

1 Coding Value Function Iteration

Now that we have some code for discretizing the state space of income (see recitation 3 notes), we can move onto the Value Function iteration.

1.1 Defining parameters and intuition

First let's define some parameters we will need in calculating the value of each state, namely the time discount factor β, depreciation constant δ, substitution between labor and capital α and risk aversion constant γ.

```
beta = 0.98 # discount factor
delta = 0.05 # depreciation factor
alpha = 0.4 # substitution of labor and capital
gamma = 2 # risk aversion constant
```

In addition to these parameters, we will need to discretize the state space of the savings state and control variable k. Say that $0 \leq k \leq 40$ with 200 points in this grid.

```
klo = 0 # min k value
khi = 40 # max k value
nk = 200 # number of grid points over space of k values
kgrid = np.linspace(klo, khi, nk) # create evenly spaced grid
```

Then, according to the algorithm for Value function iteration, it requires 4 steps (see recitation 2 notes):

1. Choose an initial guess V_0
2. Set iteration counter $j = 0$ and tolerance $\epsilon > 0$
3. Main step: on iteration j given values of V_j update V_{j+1}

$$V_{j+1}(k) = \max_{c,k'} \frac{c^{1-\gamma}}{1-\gamma} + \beta V_j(k')$$

$$\text{s.t. } c + k' = zk^{\alpha} + (1-\delta)k$$

For this recitation I'll set up the program with Cobb Douglas production function and utility given by $\frac{c^{1-\gamma}}{1-\gamma}$ as shown above.

4. Continue iterations on the main step until $|V_{j+1} - V_j| < \epsilon$

1.2 Taking the algorithm to Python

For the first step, it doesn't matter what the initial guess is (see notes on contraction mapping) so let's define it as 0. Note here that the value function V is a function of the state variables $V(k, z)$

```
Vp = np.zeros((nk, n)) # this is the next period Value function
V = Vp.copy() # this is the current period Value function
cpol = Vp.copy() # policy function for consumption
kpol = Vp.copy() # policy function for capital
ikpol = Vp.copy() # policy function for savings
```

For step number 2, we can pick a small tolerance (1^{-6}) an initial difference in the Value functions, and initial value for the iteration counter

```
dist = 1 # initial difference in Value functions
eps = 1e-6 # tolerance
iter = 1 # counter for number of iterations
```

Now we will start the Main Step. For this we will use a while loop, which continues to run so long as a condition is true: if the difference in value functions is greater than the tolerace (step 4).

```
while (dist>eps):
    #going to put loops here
```

The next piece of this iteration is to search through the possible combinations of state values of k, choice values of k', and exogenous values of technology z[1]

```
while (dist>eps):
    for iz in range(n):
        for ik in range(nk):
            for ikp in range(nk):
                #going to put some code here
```

What we want to do then with these possible combinations is to calculate consumption c based on the budget constraint, then calculate the Value function based on Step 3 above, and save the maximum values to build the policy functions as the optimal response to the state variables: $V(k, z)$, $c(k, z)$, $k'(k, z)$

```
# loop over exogenous states
```

[1]Note that in Problem Set 4 is exogenous component is wage instead of technology, but these function similarly

```
for iz in range(n):
    # loop over possible state values for savings for t period
    for ik in range(nk):
        # pick some infeasible value to compare against
        Vmax = -999999
        imax = -999999
        # loop over possible control values for savings for t+1 period
        for ikp in range(nk):
            # define consumption based on budget constraint for SPP
            c = kgrid[ik] * (1 - delta) +
                state_space[iz]*kgrid[ik]**alpha - kgrid[ikp]
            if c > 0:
                # define utility function
                U = c**(1-gamma)/(1-gamma) + \
                    beta * transtition_matrix[iz, :] @ Vp[ikp, :]
                # find and record the maximum value of the state
                if U > Vmax:
                    Vmax = U
                    imax = ikp
                    cmax = c
                    kpmax = kgrid[ikp]
        # record these maximum values over the loop
        cpol[ik, iz] = cmax
        kpol[ik, iz] = kpmax
        ikpol[ik, iz] = imax
        V[ik, iz] = Vmax
```

Putting it all together plus adding some print statements to track the progress of the Main Step:

```
start_time = time.time()
while (dist > eps):
    # loop over possible income values (discretized via Tauchin's
        method)
    for iz in range(n):
        # loop over possible state values for savings for t period
        for ik in range(nk):
            # pick some infeasible value to compare against
            Vmax = -999999
            imax = -999999
            # loop over possible control values for savings for t+1
                period
            for ikp in range(nk):
                # define consumption based on budget constraint based
                    on SPP
                c = kgrid[ik] * (1 - delta) +
                    state_space[iz]*kgrid[ik]**alpha - kgrid[ikp] #
                    note you'll need to change the BC here in PS4
                if c > 0:
```

```
                    # define utility function
                    U = c**(1-gamma)/(1-gamma) + \
                        beta * transtition_matrix[iz, :] @ Vp[ikp, :]
                    # find and record the maximum value of the state
                    if U > Vmax:
                        Vmax = U
                        imax = ikp
                        cmax = c
                        kpmax = kgrid[ikp]
                if imax < 0:
                    print('WARNING: maximization step failed!')
                    exit(1)
                elif imax == nk:
                    print('WARNING: grid is too tight!')
                    exit(1)
                # record these maximum values over the loop
                cpol[ik, iz] = cmax
                kpol[ik, iz] = kpmax
                ikpol[ik, iz] = imax
                V[ik, iz] = Vmax
        # continue while loop until the difference in Value functions
            has converged
        dist = np.max(np.abs(V - Vp))
        # print progress through loop
        print('Iter: %5d, Dist: %12.8f' % (iter, dist))
        Vp = V.copy()
        iter += 1


    end_time = time.time()
    execution_time = end_time - start_time
    print("Execution time:", execution_time, "seconds")
```

Once you have the value and policy functions, you can plot these over the space of capital and income, if you need help with this let me know!

1.3 Simulating a path of exogenous shocks

Then we want to simulate a path of exogenous shocks, and estimate the mean level of capital. The steps to do so include:

1. Pick an initial income level based on the 5 discretized state spaces
2. Check if a random variable which varies between 0 and 1 would be located in cumulative sum of the transition matrix... this implicitly selects the new level of income conditional on the current level of income
3. Do this for the entire path
4. Evaluate the optimal savings based on this sequence of income levels

5. Calculate the mean of this

Step 1 is to pick an initial level of income and we can guess an initial level of savings as well

```
ik0 = 20 # start with a guess of k0
iy0 = 2 # starting level of income
```

Then we specify the length of the simulation and generate a random variable between 0 and 1

```
nsim = 100000 # length of simulation
U = np.random.rand(nsim) # a vector of random numbers between 0 and 1
```

Step 2 is to check where the random variable falls in the cumulative sum of the transition matrix and step 3 is to record this value

```
F = np.cumsum(transition_matrix, axis=1) # take the cumulative sum
    over transition matrix
sim = np.zeros(len(U)) # create a blank random path of incomes
sim[0] = iy0 # set initial value of path
for i in range(1, len(U)): # loop over path to set subsequent
    incomes based on transition matrix
    sim[i] = np.sum(U[i] > F[int(sim[i-1]),:])
```

Then for step 4 we can use the (discretized) policy functions to find the optimal response of savings

```
savings = np.zeros(nsim)
for i in range(nsim): # record responses to these incomes based on
    policy functions
    iy = int(sim[i])
    savings[i] = kpol[ik0,iy]
    ik0 = int(savings[i])

# burn the first 10%
savings = savings[int(0.1*nsim):]
kMean = np.mean(savings) # calculate the mean
```